Effect of Thread Count and SIMD Vectorisation on Dense Matrix Multiplication: an OpenMP Study on a Hybrid Multicore CPU

Javier Bolívar García-Izquierdo
Assignment 3

Big Data
Grado en Ciencia e Ingeniería de Datos, ULPGC
2026–2027

Abstract

This report studies the effect of OpenMP thread count and compiler auto-vectorisation on dense double-precision matrix multiplication. Four implementations were measured on one Windows 11 system: sequential scalar, parallel scalar, sequential SIMD-enabled, and parallel SIMD-enabled. The final clean session contains 1484 raw measurements and 66 measured configurations. The *i-k-j* kernel uses row-major matrices, and only the multiplication is timed. At $n = 1024$, parallel SIMD with $p = 12$ has a same-level speedup of 6.40 and efficiency of 0.533 relative to sequential SIMD. At $n = 4096$, the corresponding values are 2.93 and 0.244, based on three repetitions. These values describe one hybrid CPU and one session; no universal performance claim is made.

1 Introduction and objective

Dense matrix multiplication is a useful shared-memory experiment because its computation is regular and its output rows can be distributed between threads. This assignment investigates the following research question: *How does thread count affect the runtime, speedup, and efficiency of dense matrix multiplication, and how much additional performance does SIMD auto-vectorisation provide?*

The following statements are hypotheses, not results:

- Speedup may increase approximately linearly up to the physical-core limit and then flatten or decrease when more logical threads are used.
- Small matrices may have lower efficiency because fixed overheads are significant.
- SIMD may improve both sequential and parallel implementations.

2 Background

OpenMP expresses parallel work over shared memory [2]. In this experiment, the outer matrix-row loop is distributed with a static schedule. Amdahl's law describes the limit imposed by work that does not scale with the number of threads [1]. The Roofline model relates attainable arithmetic throughput to arithmetic intensity and memory bandwidth [3].

Let T_{seq} denote the median time of a matching sequential baseline and let T_p denote the median time with p threads. Speedup and efficiency are defined as

$$S(p) = \frac{T_{\text{seq}}}{T_p}, \qquad E(p) = \frac{S(p)}{p}.$$

The statistic used for time is the median and variability is reported with the interquartile range (IQR). Same-level speedup uses `seq_scalar` for `par_scalar` and `seq_simd` for `par_simd`. Combined speedup always uses `seq_scalar`.

SIMD vectorisation evaluates independent operations in vector registers. The *i*-*k*-*j* loop order makes the innermost j loop contiguous in memory. This gives the compiler a regular loop to vectorise while keeping the mathematical algorithm identical between builds.

3 Methodology

3.1 Algorithm and variants

The matrices are contiguous row-major arrays of `double`. The kernel computes

$$C_{ij} \mathrel{+}= A_{ik}B_{kj}$$

with loop order *i*-*k*-*j*. The input generator is xorshift64 with seed `123456789`; generated values are uniform in $[0, 1)$. Allocation, generation, and clearing C are outside the timed interval. A checksum is computed after timing.

The exact builds are:

`seq_scalar` `-std=c11 -O3 -fno-tree-vectorize -fno-tree-slp-vectorize -ffp-contract=off`.

`par_scalar` The sequential scalar flags plus `-fopenmp`.

`seq_simd` `-std=c11 -O3 -march=native -ffp-contract=off`.

`par_simd` The sequential SIMD flags plus `-fopenmp`.

No BLAS, NumPy, fast-math, or optimised library is used in the timed kernel. The compiler is GCC 16.1.0, and the flag meanings are documented in the GCC manual [4].

3.2 Hardware and software

The recorded system is Windows 11, an Intel Core i5-13420H, 8 physical cores, 12 logical processors, and 15.64 GB of RAM. Its topology is hybrid: 4 P-cores with Hyper-Threading and 4 E-cores. The compiler is MSYS2 UCRT64 GCC 16.1.0. Python is version 3.13.12; the recorded analysis environment also contains NumPy 2.2.6, pandas 2.3.3, matplotlib 3.10.6, and psutil 7.1.0. The power plan recorded in the system information was Equilibrado and AC power was reported as true.

3.3 Timing protocol

On Windows, timing uses `QueryPerformanceCounter`; on other platforms the source uses `clock_gettime(CLOCK_MONOTONIC)`. The timer includes only the multiplication kernel. Input generation, allocation, zero-initialisation, and checksum calculation are outside it.

The tested sizes are $n \in \{256, 512, 1024, 2048, 4096\}$. Sizes through $n = 1024$ use two warm-ups and 30 measured repetitions. The $n = 2048$ cases use one warm-up and 10 repetitions. The $n = 4096$ cases use one warm-up and three repetitions. For $n = 256$ and $n = 512$, ten multiplications are batched and the CSV stores the average time per multiplication.

Parallel variants use $p \in \{1, 2, 4, 6, 8, 12\}$ for the regular sizes, plus $p = 24$ at $n = 1024$. Sequential variants use $p = 1$. At $n = 4096$, the reduced coverage contains sequential scalar and SIMD, parallel SIMD at $p \in \{1, 4, 8, 12\}$, and parallel scalar at $p \in \{1, 12\}$. A 120-second cap applies to each multiplication.

The pinning test at $n = 1024$ compared 30 repetitions without explicit CPU affinity against 30 repetitions pinned to logical CPU 0. Unpinned `seq_simd` had median 0.285 s and IQR 0.0362

s; CPU-0-pinned `seq_simd` had median 0.260 s and IQR 0.0169 s. The final session therefore pins sequential and $p = 1$ runs to CPU 0; runs with $p > 1$ are not pinned. The environment sets `OMP_NUM_THREADS`, `OMP_PROC_BIND=close`, and `OMP_PLACES=cores`. MinGW `libgomp` reported that affinity was not supported and displayed an empty `OMP_PLACES`.

Table 1: Configuration sets represented in the final clean session.

Size	Sequential variants	Parallel thread counts	Repetitions
256, 512, 1024	scalar and SIMD, $p = 1$	$p = 1, 2, 4, 6, 8, 12$; $p = 24$ at 1024	30
2048	scalar and SIMD, $p = 1$	$p = 1, 2, 4, 6, 8, 12$	10
4096	scalar and SIMD, $p = 1$	SIMD $p = 1, 4, 8, 12$; scalar $p = 1, 12$	3

3.4 Correctness and memory

Correctness tests cover identity, zero, a hand-computed 3×3 product, and NumPy reference results for $n = 64$, $n = 128$, and $n = 256$. The comparison uses atol = rtol = 10^{-9}. Sequential and parallel result checksums match exactly in the tested configurations. Each thread writes only its assigned rows of C, so the outer parallel loop has no write race.

On Windows, peak memory uses `GetProcessMemoryInfo` and `PeakWorkingSetSize`. The theoretical array-storage metric is $3 \cdot 8 \cdot n^2$ bytes. These are different quantities. The memory budget was 2 GB.

3.6 Vectorisation evidence

The saved GCC report for `seq_simd` states: "`matmul.c:57:26: optimized: loop vectorized using 16 byte vectors and unroll factor 2`". The saved `par_simd` disassembly contains the following hot-loop excerpt:

```
vmulpd (%rdx,%r8,1),%ymm1,%ymm0
vaddpd (%rax,%r8,1),%ymm0,%ymm0
vmovupd %ymm0,(%rax,%r8,1)
```

The vectoriser report also records 32-byte vectors in the SIMD build. The assembly uses AVX instructions with YMM registers; the evidence files do not identify a narrower or wider named SIMD extension beyond these instructions.

3.7 Discarded runs

The first discarded run had the laptop lid closed while a game was running. The second used the coarse OpenMP timer observed in the parallel builds. The third preliminary run applied the 120-second limit to the whole process instead of to each multiplication. All are retained under `results/archive/` and excluded from the final analysis.

4 Results

All values in this section come from the final `summary.csv`, `raw_measurements.csv`, or `sanity_report.md`. Times are seconds per multiplication. Error values are IQR and r is the number of measured repetitions. Sentences in this section are descriptive only.

Table 2: Median time in seconds for every tested thread count. A dash means that the configuration was not part of the final session. Values are rounded to three significant digits.

Variant	n	$p = 1$	$p = 2$	$p = 4$	$p = 6$	$p = 8$	$p = 12$	$p = 24$
seq_scalar	256	0.00737	–	–	–	–	–	–
seq_scalar	512	0.0585	–	–	–	–	–	–
seq_scalar	1024	0.501	–	–	–	–	–	–
seq_scalar	2048	5.36	–	–	–	–	–	–
seq_scalar	4096	48.7	–	–	–	–	–	–
seq_simd	256	0.00226	–	–	–	–	–	–
seq_simd	512	0.0275	–	–	–	–	–	–
seq_simd	1024	0.262	–	–	–	–	–	–
seq_simd	2048	3.77	–	–	–	–	–	–
seq_simd	4096	33.2	–	–	–	–	–	–
par_scalar	256	0.00762	0.00420	0.00194	0.00184	0.00150	0.00123	–
par_scalar	512	0.0581	0.0296	0.0144	0.0140	0.0113	0.00916	–
par_scalar	1024	0.503	0.268	0.145	0.123	0.102	0.0749	0.0776
par_scalar	2048	5.34	3.30	1.77	1.46	1.18	0.964	–
par_scalar	4096	47.0	–	–	–	–	12.3	–
par_simd	256	0.00223	0.00117	0.000589	0.000638	0.000724	0.000646	–
par_simd	512	0.0275	0.0122	0.00665	0.00561	0.00514	0.00397	–
par_simd	1024	0.268	0.127	0.0635	0.0614	0.0486	0.0409	0.0404
par_simd	2048	3.78	2.20	1.17	0.970	0.901	0.767	–
par_simd	4096	33.3	–	10.9	–	11.5	11.3	–

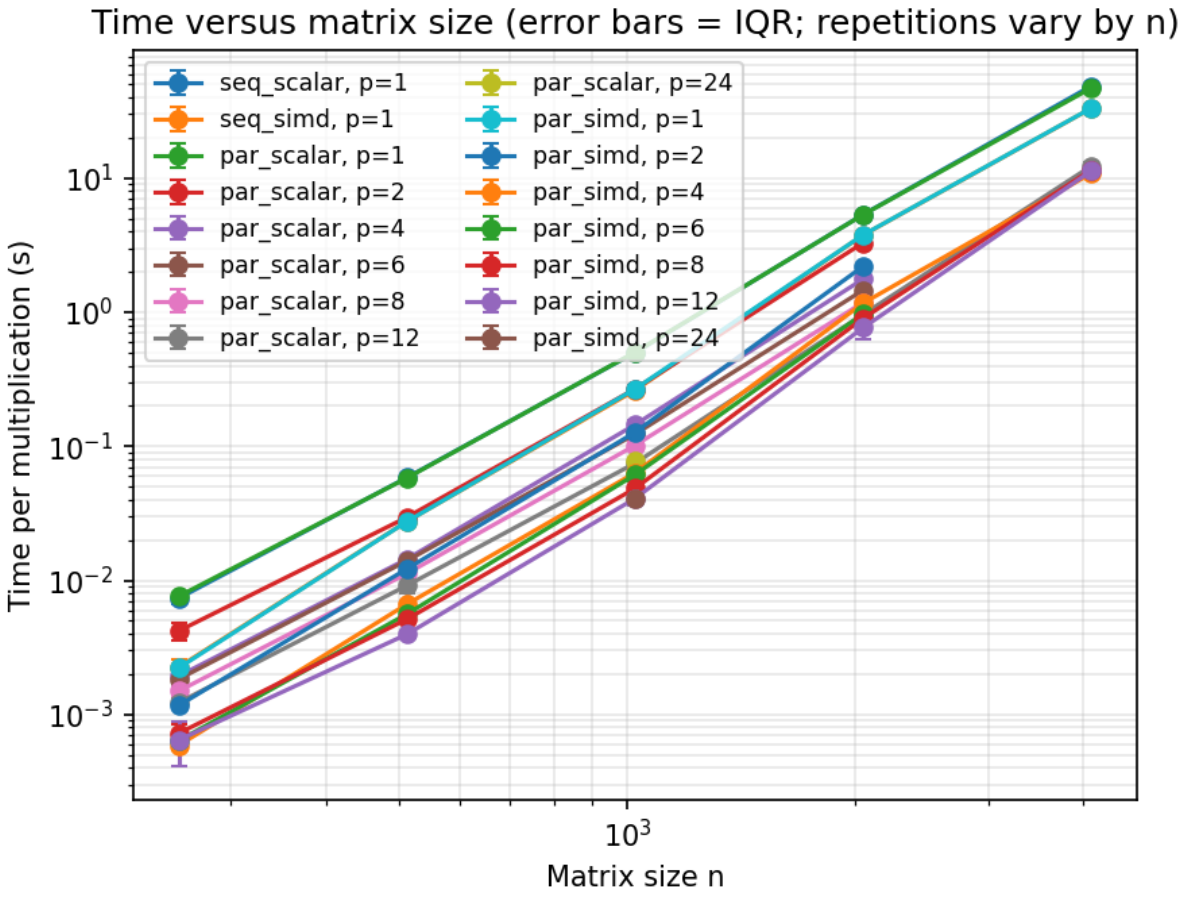


Figure 1: Median time per multiplication versus n on log–log axes for $p = 1$, $p = 4$, and $p = 12$; error bars are IQR.

Table 3: Same-level speedup/efficiency for every tested thread count, shown as $S(p)/E(p)$. Scalar rows use `seq_scalar`; SIMD rows use `seq_simd`. Values are rounded to three significant digits.

Variant	n	$p = 1$	$p = 2$	$p = 4$	$p = 6$	$p = 8$	$p = 12$	$p = 24$
`par_scalar`	256	0.966/0.966	1.75/0.876	3.80/0.951	4.01/0.668	4.92/0.615	6.00/0.500	–
`par_scalar`	512	1.01/1.01	1.98/0.989	4.05/1.01	4.18/0.696	5.20/0.650	6.39/0.532	–
`par_scalar`	1024	0.995/0.995	1.87/0.934	3.45/0.863	4.07/0.678	4.93/0.616	6.69/0.557	6.45/0.269
`par_scalar`	2048	1.00/1.00	1.63/0.813	3.03/0.758	3.66/0.611	4.52/0.565	5.56/0.463	–
`par_scalar`	4096	1.04/1.04	–	–	–	–	3.96/0.330	–
`par_simd`	256	1.01/1.01	1.93/0.967	3.84/0.959	3.54/0.590	3.12/0.390	3.50/0.291	–
`par_simd`	512	1.00/1.00	2.26/1.13	4.14/1.04	4.91/0.818	5.36/0.670	6.94/0.578	–
`par_simd`	1024	0.977/0.977	2.06/1.03	4.12/1.03	4.26/0.710	5.38/0.673	6.40/0.533	6.47/0.270
`par_simd`	2048	0.997/0.997	1.71/0.855	3.22/0.806	3.88/0.647	4.18/0.522	4.91/0.409	–
`par_simd`	4096	0.998/0.998	–	3.06/0.765	–	2.88/0.360	2.93/0.244	–

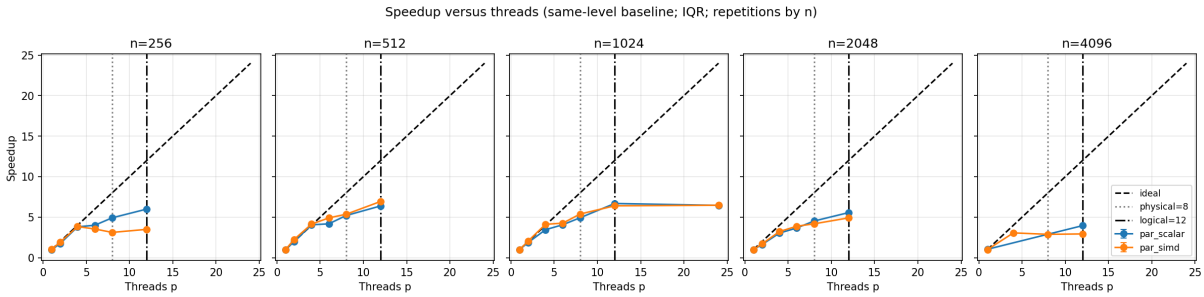


Figure 2: Same-level speedup versus p in two rows of size panels. The baseline is sequential scalar for scalar and sequential SIMD for SIMD. The ideal line and the physical and logical processor markers are shown; error bars represent IQR-derived uncertainty.

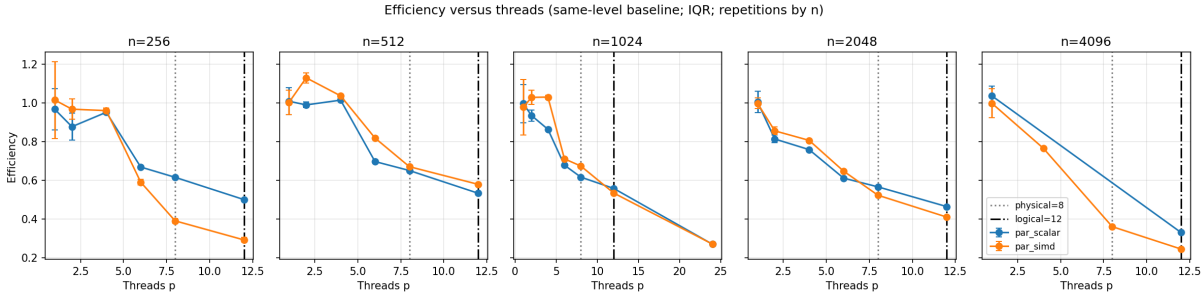


Figure 3: Same-level efficiency versus p in two rows of size panels, using the same baselines as the speedup figure. Error bars represent IQR-derived uncertainty.

4.1 Runtime

4.2 Speedup and efficiency

4.3 OpenMP $p = 1$ overhead

Table 4: OpenMP $p = 1$ speedup and efficiency relative to the matching sequential baseline.

Variant	n	$S(1)$	$E(1)$	IQR (s)	r
`par_scalar`	256	0.966	0.966	0.000106	30
`par_simd`	256	1.01	1.01	0.000140	30
`par_scalar`	512	1.01	1.01	0.00160	30
`par_simd`	512	1.00	1.00	0.000970	30
`par_scalar`	1024	0.995	0.995	0.0235	30
`par_simd`	1024	0.977	0.977	0.0186	30
`par_scalar`	2048	1.00	1.00	0.110	10
`par_simd`	2048	0.997	0.997	0.0434	10
`par_scalar`	4096	1.04	1.04	1.52	3
`par_simd`	4096	0.998	0.998	1.21	3

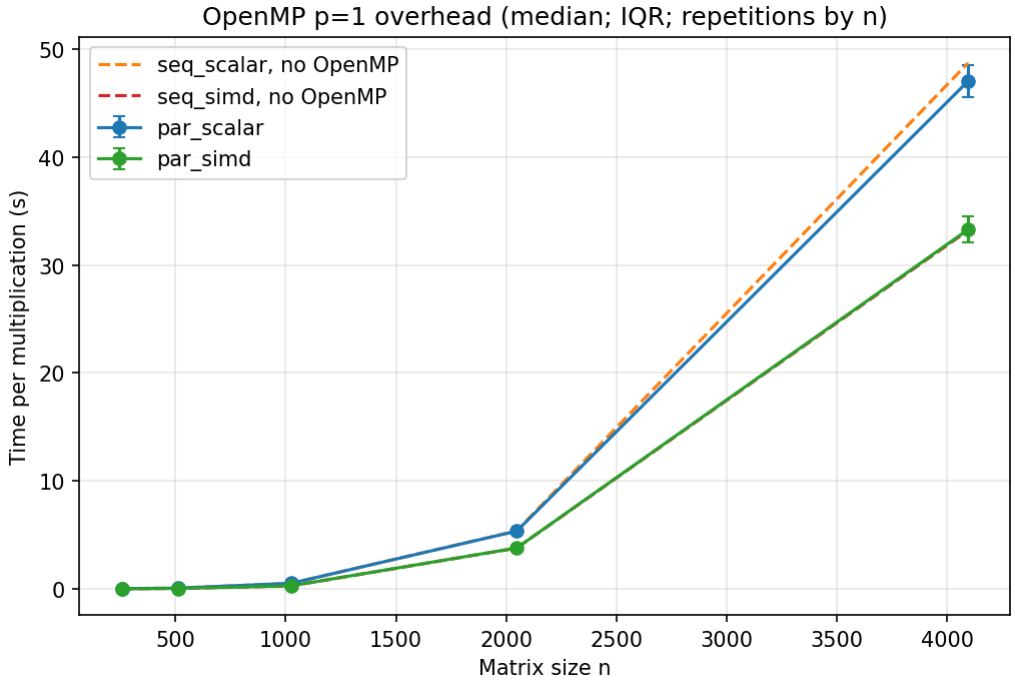


Figure 4: Runtime of parallel variants at $p = 1$ and their matching sequential baselines. Error bars are IQR.

4.4 Vectorisation and GFLOP/s

4.5 Memory, oversubscription, and $n = 4096$

The $n = 4096$ measurements use three repetitions and the reduced configuration set stated in the methodology. The sequential scalar median is 48.7 s, the sequential SIMD median is 33.2 s, and the parallel SIMD $p = 12$ median is 11.3 s.

Table 5: Two-by-two vectorisation view at $n = 2048$ and $n = 4096$.

Variant	n	p	Median (s)	IQR (s)	r
`seq_scalar`	2048	1	5.36	0.185	10
`seq_simd`	2048	1	3.77	0.0619	10
`par_scalar`	2048	12	0.964	0.0402	10
`par_simd`	2048	12	0.767	0.136	10
`seq_scalar`	4096	1	48.7	0.957	3
`seq_simd`	4096	1	33.2	1.29	3
`par_scalar`	4096	12	12.3	0.683	3
`par_simd`	4096	12	11.3	1.31	3

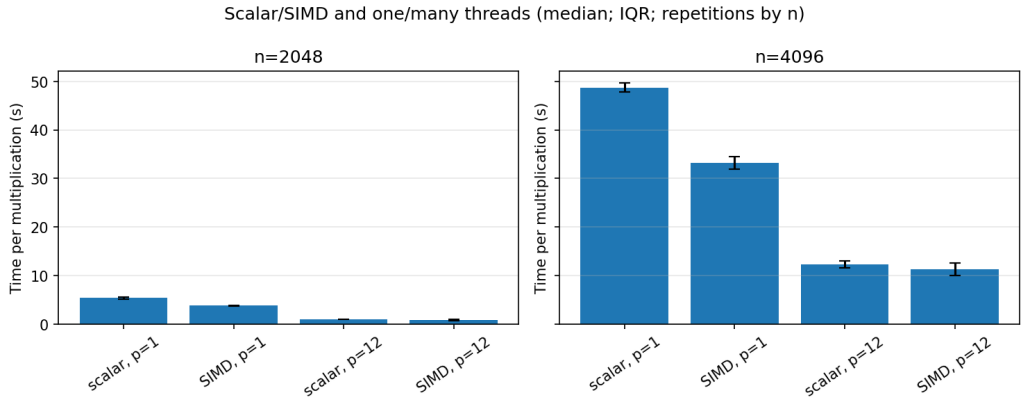


Figure 5: Scalar/SIMD and one/12-thread runtimes at $n = 2048$ and $n = 4096$. Bars are medians and error bars are IQR.

Table 6: Sequential GFLOP/s calculated as $2n^3/T_{\text{seq}}/10^9$.

Variant	n	GFLOP/s	IQR (s)	r
`seq_scalar`	256	4.56	0.000651	30
`seq_simd`	256	14.9	0.000314	30
`seq_scalar`	512	4.59	0.00249	30
`seq_simd`	512	9.75	0.000780	30
`seq_scalar`	1024	4.29	0.0260	30
`seq_simd`	1024	8.21	0.0187	30
`seq_scalar`	2048	3.21	0.185	10
`seq_simd`	2048	4.56	0.0619	10
`seq_scalar`	4096	2.82	0.957	3
`seq_simd`	4096	4.14	1.29	3

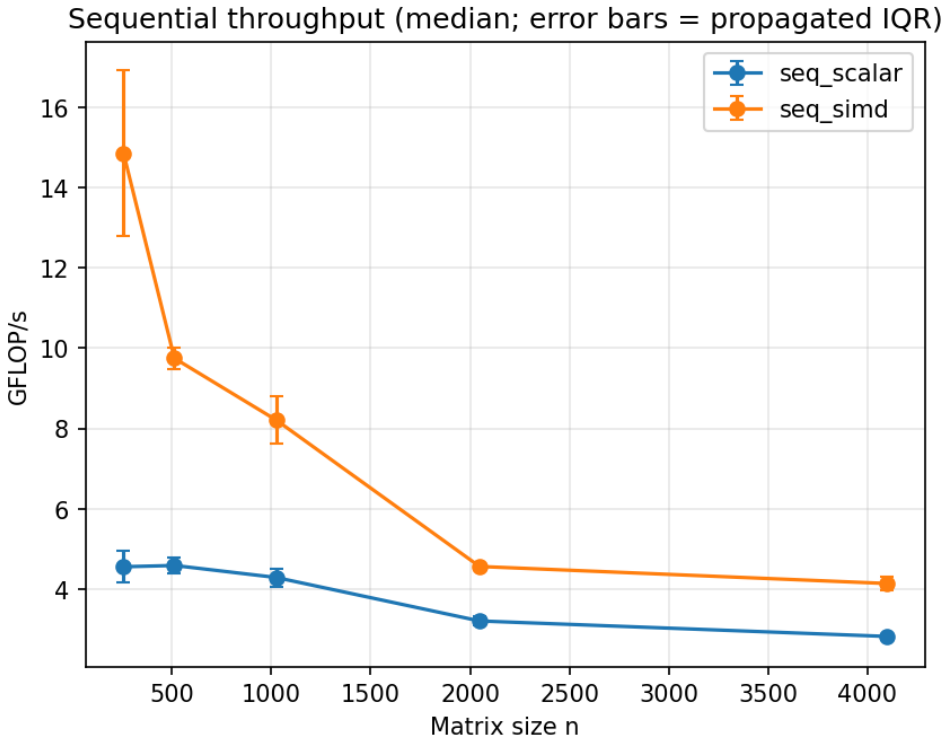


Figure 6: Sequential GFLOP/s versus n, calculated from median time. Error bars represent propagated IQR.

Table 7: Peak working set measured with Windows `PeakWorkingSetSize`, compared with theoretical array storage. For $n = 256$ and $n = 512$, batch mode allocates one separate result buffer per batched multiplication, so the measured process peak includes these buffers and is not the storage of only three arrays.

n	Peak median (MiB)	Peak IQR (MiB)	Theoretical (MiB)
256	9.71	0	1.50
512	27.7	0	6.00
1024	27.7	0	24.0
2048	99.7	0	96.0
4096	388	0.102	384

Table 8: Oversubscription comparison at $n = 1024$. The ratio is T_{12}/T_{24}; $p = 24$ is the oversubscribed configuration.

Variant	T_{12}/T_{24}	T_{12} (s)	T_{24} (s)	r
`par_scalar`	0.965	0.0749	0.0776	30
`par_simd`	1.01	0.0409	0.0404	30

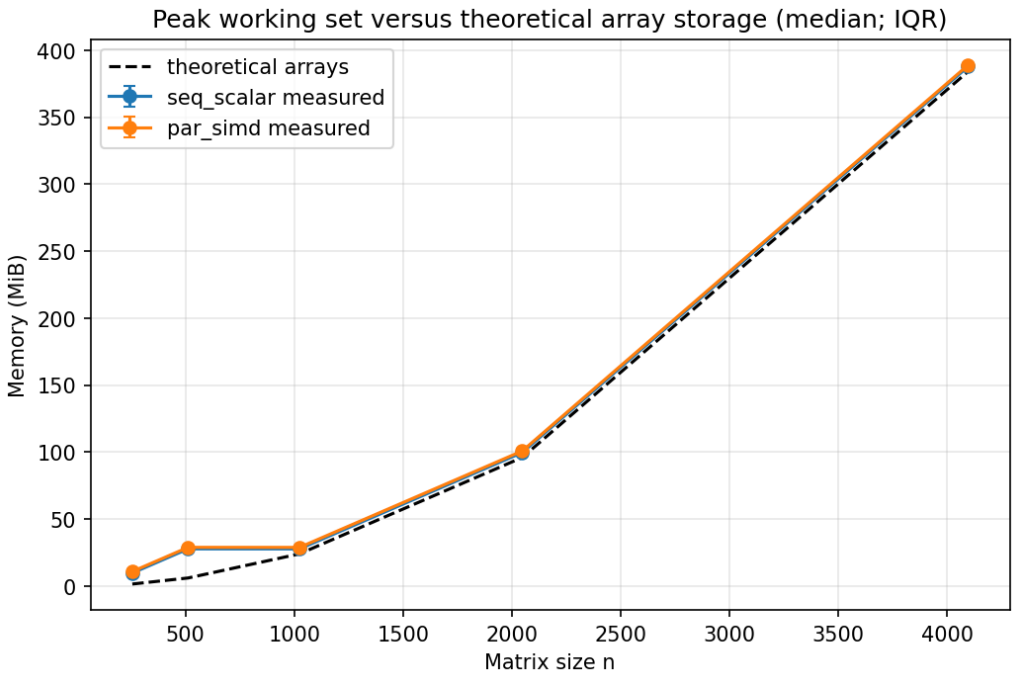


Figure 7: Measured peak working set and theoretical three-array storage. Measured points are medians with IQR error bars.

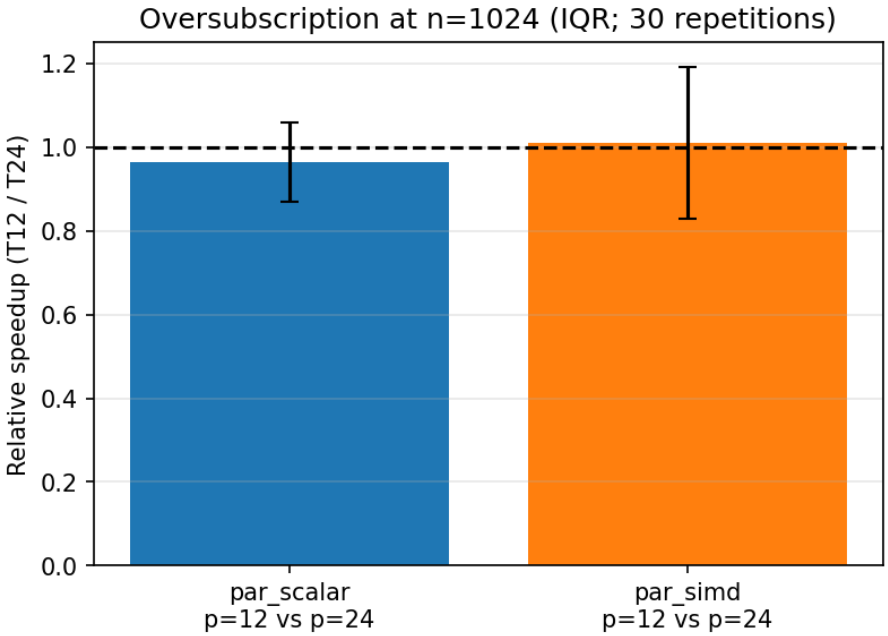


Figure 8: Relative $p = 12$ versus $p = 24$ speed at $n = 1024$ for the parallel variants. Error bars are based on IQR.

5 Discussion

TODO (student). No interpretation is supplied here.

- Where does speedup stop increasing, and how might the hybrid topology relate to that point?
- What does the $p = 1$ overhead show for scalar and SIMD variants?
- What does the 2x2 vectorisation view show at the two largest sizes?
- Do the GFLOP/s and memory figures support a cache or memory-bound explanation?
- How should the IQR and sanity-report flags affect confidence?

5.1 Limitations

The processor is hybrid, with 4 P-cores with Hyper-Threading and 4 E-cores; thread counts above the tested homogeneous subset mix core types. Sequential and $p = 1$ runs are pinned to CPU 0, while runs with $p > 1$ are not pinned. MinGW `libgomp` reported that `OMP_PROC_BIND` and `OMP_PLACES` could not enforce affinity. The $n = 4096$ set is reduced and has three repetitions. The experiment uses one machine and one session; thermal state and background activity were not fully controlled. The kernel is not cache-blocked, so memory-bound behaviour was not tested directly. Peak working set and theoretical array storage are different metrics.

6 Conclusions

TODO (student).

- Which hypotheses are supported or rejected by the measured data?
- What is the measured contribution of SIMD relative to threading?
- Which thread counts are most defensible for this hybrid CPU?
- Which conclusions remain limited by the protocol and reduced $n = 4096$ coverage?

7 AI usage disclosure

An AI coding agent (Google Antigravity) was used for code generation, debugging, benchmark-driver development, analysis scripts, figure generation, and report scaffolding. I validated the correctness tests and measurements and am responsible for the final content, interpretation, and submission.

8 References

References

[1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[2] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, Nov. 2021.

[3] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[4] Free Software Foundation, *Using the GNU Compiler Collection (GCC), for GCC version 16.1.0*, GCC documentation, 2026. Available: https://gcc.gnu.org/onlinedocs/gcc-16.1.0/gcc/

Repository: `https://github.com/Javi05x/INDIVIDUAL_ASSIGMENT3.git`. The reproducibility materials are the C source, Python driver and analysis scripts, final raw CSV, summary CSV, figures, vectorisation evidence, README, and this report.